

Department of Computer Science & Engineering

Microprocessor & Computer Architecture

UE24CS251B

MPCA Orange Problem

Name: Dhriti Kamani	SRN: PES1UG24AM085	Section: B
A	Question A: 5. A. Scenario: WhatsApp Sticker Disaster (6 Marks) A student has N different WhatsApp stickers saved on their phone and wants to spam their friend. The student considers two different situations: 1. Sticker Order Matters Mode The friend gets annoyed if the same stickers arrive in a different order. 2. Sticker Order Doesn't Matter Mode The friend only cares about which stickers are sent, not the order. The value of N and the number of stickers sent R are stored in memory. Given Data N = 6 R = 3 Problem Statement Write a RISC-V assembly program using RARS to: 1. Calculate the total number of different sticker messages possible when order matters. 2. Calculate the total number of different sticker messages possible when order does not matter. 3. Store both results in memory. 4. Display both values using system calls. Constraints Use loops for all calculations. Use only arithmetic and conditional branch instructions. Do not use any precomputed formulas directly. Do not use pseudo instructions beyond standard RISC-V. The program must run correctly in RARS. Code with comments: <pre>.data N: .word 6 R: .word 3 perm_res: .word 0 comb_res: .word 0 .text .globl main main: # Load N and R la t0, N lw t1, 0(t0) # t1 = N la t0, R lw t2, 0(t0) # t2 = R # ----- PERMUTATION ----- addi t3, x0, 1 # result = 1 add t4, t1, x0 # current = N add t5, t2, x0 # counter = R perm_loop:</pre>	

```
beq t5, x0, perm_done
mul t3, t3, t4
addi t4, t4, -1
addi t5, t5, -1
jal x0, perm_loop

perm_done:

# store permutation result
la t0, perm_res
sw t3, 0(t0)

# ----- FACTORIAL OF R -----
addi t4, x0, 1    # factorial result
add t5, t2, x0    # counter = R

fact_loop:
beq t5, x0, fact_done
mul t4, t4, t5
addi t5, t5, -1
jal x0, fact_loop

fact_done:

# ----- COMBINATION -----
div t6, t3, t4    # combination result

la t0, comb_res
sw t6, 0(t0)

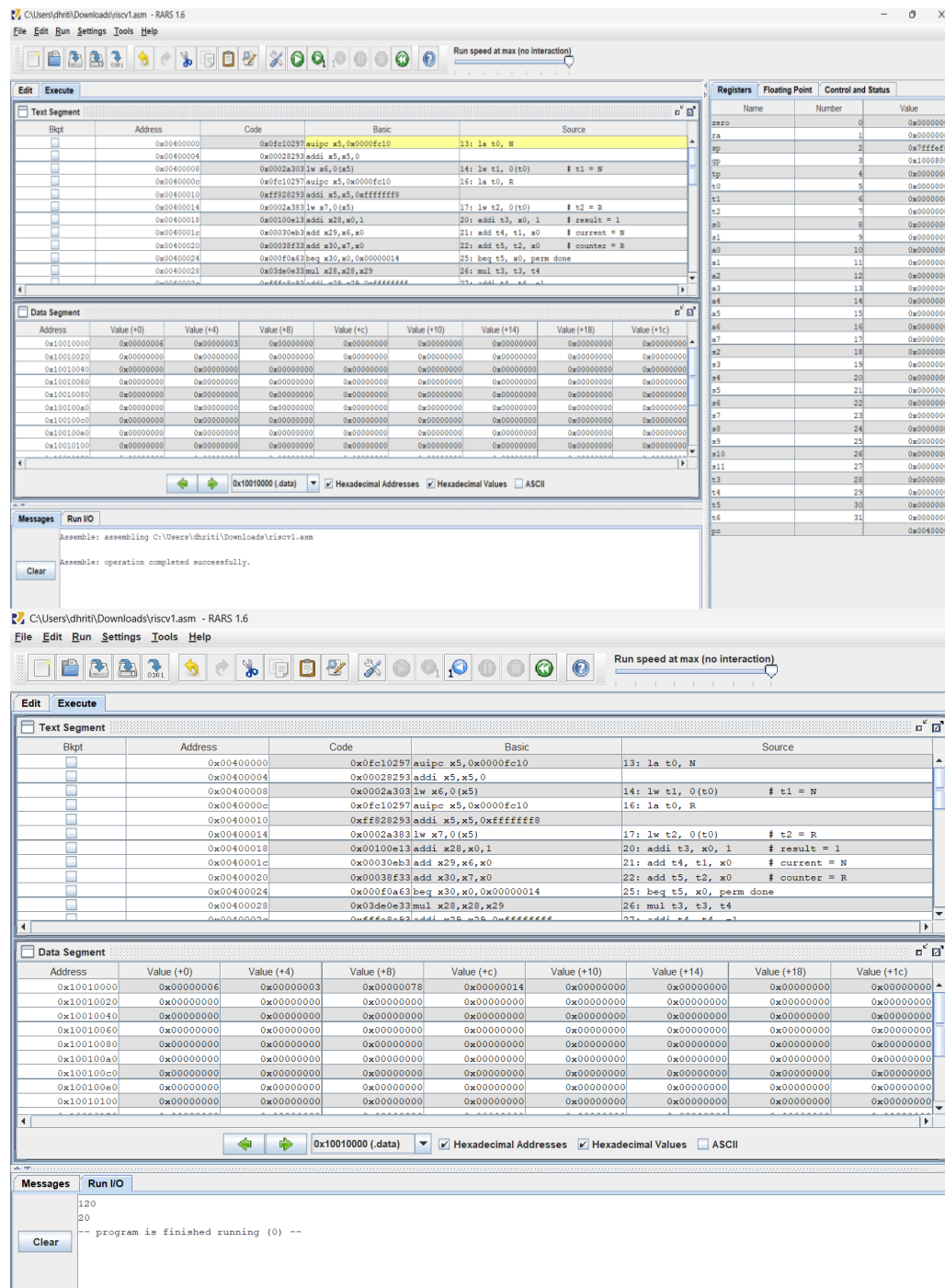
# ----- PRINT PERMUTATION -----
addi a7, x0, 1
add a0, t3, x0
ecall

# newline
addi a7, x0, 11
addi a0, x0, 10
ecall

# ----- PRINT COMBINATION -----
addi a7, x0, 1
add a0, t6, x0
ecall

# exit
addi a7, x0, 10
ecall
```

Screenshot of the output:



Text Segment

Address	Code	Basic	Source
0x00400000	0x0fc10297 luiip x5,0x0000fc10	13: la t0, N	
0x00400004	0x00028293 addi x5,x5,0	14: la t0, R	
0x00400008	0x0002a303 lw x6,0(x5)	14: lw t1, 0(t0) # t1 = N	
0x0040000c	0x0fc10297 luiip x5,0x0000fc10	16: la t0, R	
0x00400010	0x00028293 addi x5,x5,0xffffffff	17: lw t2, 0(t0) # t2 = R	
0x00400014	0x0002a303 lw x7,0(x5)	20: addi t3, x0, 1 # result = 1	
0x00400018	0x00100e13 addi x28,x0,1	21: add t4, t1, x0 # current = N	
0x0040001c	0x00030eb3 add x29,x6,x0	22: add t5, t2, x0 # counter = R	
0x00400020	0x00038f33 add x30,x7,x0	25: beq t5, x0, perm done	
0x00400024	0x000f0a63 beq x30,x0,0x00000014	26: mul t3, t3, t4	
0x00400028	0x03de0e33 mul x28,x28,x29	27: addi t4, t4, -1	

Data Segment

Address	Value (+0)	Value (+4)	Value (+8)	Value (+c)	Value (+10)	Value (+14)	Value (+18)	Value (+1c)
0x10010000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x10010004	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x10010008	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x1001000c	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x10010010	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x10010014	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x10010018	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x1001001c	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x10010020	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x10010024	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x10010028	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x1001002c	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x10010030	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x10010034	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x10010038	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x1001003c	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x10010040	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x10010044	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x10010048	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x1001004c	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x10010050	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x10010054	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x10010058	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x1001005c	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x10010060	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x10010064	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x10010068	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x1001006c	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x10010070	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x10010074	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x10010078	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x1001007c	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x10010080	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x10010084	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x10010088	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x1001008c	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x10010090	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x10010094	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x10010098	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x1001009c	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x100100a0	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x100100a4	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x100100a8	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x100100ac	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x100100b0	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x100100b4	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x100100b8	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x100100bc	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x100100c0	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x100100c4	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x100100c8	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x100100cc	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x100100d0	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x100100d4	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x100100d8	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x100100dc	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x100100e0	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x100100e4	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x100100e8	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x100100ec	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x100100f0	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x100100f4	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x100100f8	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x100100fc	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000

Messages

```

Assembly: assembling C:\Users\dhriti\Downloads\riscv1.asm
Assembly: operation completed successfully.

```

B Question B:

B. Scenario: Simple Data Encryption (4 Marks)

A message is stored as an integer array. Problem Statement

Write a RISC-V assembly program using RARS to:

- Add index value to each element
- Reverse the array
- Display encrypted output Constraints
- Use arithmetic instructions only
- Avoid pseudo instructions beyond standard RISC-V
- Program must execute correctly in RARS

Code with comments:

```
.data
array: .word 5,10,15,20
size: .word 4

.text
.globl main

main:
# load base address of array
la t0, array
# load size
la t1, size
lw t2, 0(t1)    # t2 = size
addi t3, x0, 0   # i = 0

# ----- ADD INDEX -----
add_index_loop:
beq t3, t2, reverse_start
slli t4, t3, 2    # offset = i*4
add t5, t0, t4
lw t6, 0(t5)      # array[i]
add t6, t6, t3    # array[i] = value + index
sw t6, 0(t5)
addi t3, t3, 1
jal x0, add_index_loop

# ----- REVERSE ARRAY -----
reverse_start:

addi t3, x0, 0
addi t4, t2, -1   # j = size-1

reverse_loop:

bge t3, t4, print_start

# load array[i]
slli t5, t3, 2
add t6, t0, t5
lw s0, 0(t6)

# load array[j]
slli t5, t4, 2
add s1, t0, t5
lw s2, 0(s1)

# swap
sw s2, 0(t6)
sw s0, 0(s1)

addi t3, t3, 1
addi t4, t4, -1

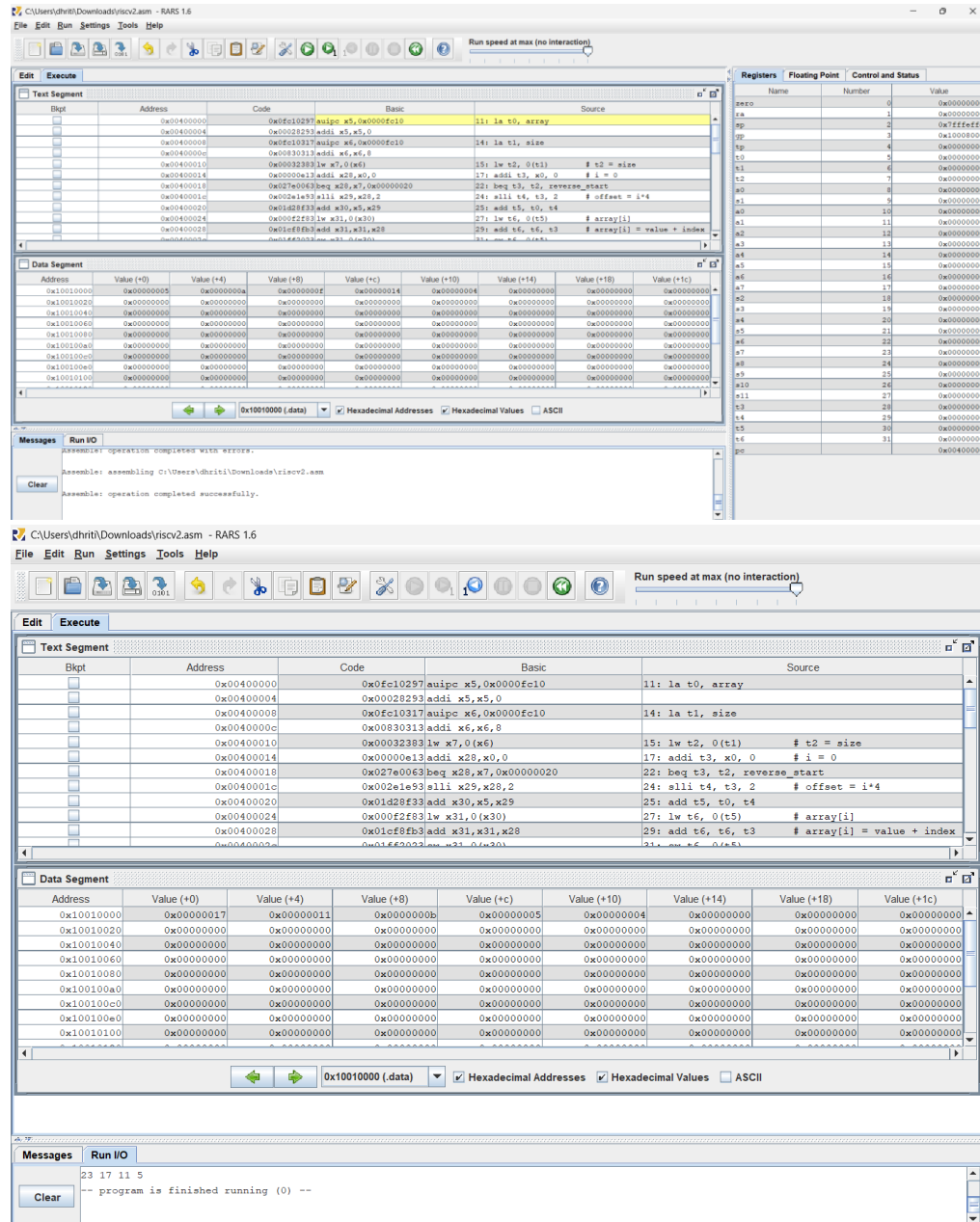
jal x0, reverse_loop

# ----- PRINT ARRAY -----
print_start:
addi t3, x0, 0
print_loop:
beq t3, t2, exit
slli t4, t3, 2
add t5, t0, t4
lw a0, 0(t5)
addi a7, x0, 1
ecall
```

```
# print space
addi a7, x0, 11
addi a0, x0, 32
ecall
addi t3, t3, 1
jal x0, print_loop
```

```
exit:
addi a7, x0, 10
ecall
```

Screenshot of the output:



The screenshot displays the RARS 1.6 IDE interface. The main window is titled "C:\Users\dhriti\Downloads\riscv2.asm - RARS 1.6". The menu bar includes File, Edit, Run, Settings, Tools, and Help. The toolbar contains various icons for file operations and execution.

The **Text Segment** panel shows the assembly code with the following instructions and comments:

Address	Code	Basic	Source
0x00400000	0x0fc10297	auipc x5, 0x0000fc10	11: la t0, array
0x00400004	0x00022293	addi x5, x5, 0	
0x00400008	0x0fc10317	auipc x6, 0x0000fc10	14: la t1, size
0x0040000c	0x00930313	addi x6, x6, 8	
0x00400010	0x00032383	lw x7, 0(x6)	15: lw t2, 0(t1) # t2 = size
0x00400014	0x00000e13	addi x28, x0, 0	17: addi t3, x0, 0 # i = 0
0x00400018	0x027e0063	beq x28, x7, 0x00000020	22: beq t3, t2, reverse_start
0x0040001c	0x002e1e93	slli x29, x28, 2	24: slli t4, t3, 2 # offset = i*4
0x00400020	0x01d28f33	add x30, x5, x29	25: add t5, t0, t4
0x00400024	0x000f2f83	lw x31, 0(x30)	27: lw t6, 0(t5) # array[i]
0x00400028	0x01cf8fb3	add x31, x31, x28	29: add t6, t6, t3 # array[i] = value + index
0x0040002c	0x01f62023	sw x31, 0(x30)	31: sw t6, 0(x30)

The **Data Segment** panel shows memory addresses and their values in hexadecimal. The registers panel shows the state of the processor registers, including zero, ra, sp, gp, t0, t1, t2, t3, t4, t5, t6, t7, a0, a1, a2, a3, a4, a5, a6, a7, s0, s1, s2, s3, s4, s5, s6, s7, s8, s9, s10, s11, s12, s13, s14, s15, s16, s17, s18, s19, s20, s21, s22, s23, s24, s25, s26, s27, s28, s29, s30, s31, and pc.

The Messages panel shows the following output:

```
Assembly operation completed with errors.
Assembly: assembling C:\Users\dhriti\Downloads\riscv2.asm
Assembly: operation completed successfully.
```